

ghactaint

Detecting and Repairing Expression Injection in GitHub Actions
IEEE Computer Society Global Student Challenge 2026 — Challenge 02 — Team 312

1. Summary

ghactaint is a static taint analyzer and automatic repair tool for expression-injection vulnerabilities in GitHub Actions. It is fully deterministic and makes **zero LLM calls**. Every claim below is measured against the 150-sample training split, not estimated.

Task	Metric	Result
1 — Detection (flow-level, exact from/to lines)	F1	0.941
	Recall	1.000 (56/56 gold flows, 0 missed)
	Precision	0.889 (7 FP, confined to 2 samples)
1 — Detection (sample-level)	Accuracy	0.987 (TP 29 / TN 119 / FP 2 / FN 0)
2 — Repair	Verification	29/29 patches pass all 4 gates
	Reference agreement	29/29 file-set match vs. reference patches

The null baseline (predict everything clean) scores 80.7% sample accuracy but **0.000** detection F1: 29 of 150 samples are vulnerable, so recall is the entire problem. No gold flow is missed.

2. Approach

A GitHub Actions expression `${{ ... }}` is substituted into the runner's shell script as *text*, before the shell parses it. If the interpolated value is attacker-controlled — a branch name, an issue title, a commit message — the attacker is writing shell source code, not supplying data. Quoting does not help: the substitution has already happened by the time the quotes are parsed.

The tool models this as an interprocedural taint problem. Sources are the 27 context paths given in `untrusted_data.csv`; sinks are `run`: scripts; propagation crosses files via `with`: bindings, step and job outputs, and environment variables.

2.1 Architecture

Module	Responsibility
<code>contexts.py</code>	The untrusted source set; wildcard and bracket canonicalization
<code>expr.py</code>	<code>\${{ }}</code> scanner with byte offsets; <code>format/ /fromJSON</code> handling
<code>resolver.py</code>	uses: reference to vendored on-disk action path
<code>loader.py</code>	YAML round-trip; block-scalar offset-to-line mapping
<code>taint.py</code>	The interprocedural taint engine (flows and consumptions)
<code>patch.py</code>	env-hoist repair and unified-diff generation
<code>verify.py</code>	Four-gate self-verification
<code>eval.py</code>	Scoring harness with tolerance sweep and null baseline

The analyzer never re-serializes YAML. Patches are surgical edits to the raw text, so untouched formatting, comments, and ordering survive byte-for-byte — a requirement for `git apply` to behave predictably.

3. Task 1 — Detection

Three parts of the flow model could not be settled by reading the specification. Each was resolved by measurement against the gold labels rather than by assumption.

3.1 The sink-line convention, resolved by sweep

For a multi-line `run:` block, gold could point at the `${{ }}` line, the `run:` key line, or the step start. All three were implemented behind a flag and swept against the real labels:

Mode	Strict (exact line)	File-only	Verdict
expr	F1 0.850	F1 0.850	strict == file-only at every tolerance
run-key	collapses	F1 0.800	degrades as tolerance tightens
step-start	collapses	F1 0.800	degrades as tolerance tightens

The signature is decisive: under `expr`, strict and file-only scores are *identical* at every tolerance, meaning every flow located on the correct file also landed on the exactly correct line. A wrong convention cannot produce that. The convention is the line of the `${{ }}` occurrence. (Scores shown are from the sweep, before the model refinements in 3.2–3.3.)

3.2 Input defaults let a component taint itself

A composite action can be the source of its own vulnerability. If an input declares a default that references untrusted context, and the caller does not bind that input, the default applies and the taint enters *inside* the action — with no untrusted expression anywhere in the calling workflow:

```
inputs:
  title:
    default: "${{ github.event.pull_request.title }}" # taint enters here
runs:
  # ... later: if [[ '${{ inputs.title }}' =~ ... ]] # sink
```

This was an entire class of false negatives, and it is invisible without the dataset: nothing in the workflow looks wrong. Modeling it moved recall from 0.893 to 1.000.

3.3 Root versus derived reporting

The specification says downstream sinks of an already-reported value need not be re-reported. The subtlety is what counts as downstream. Gold reports *four* separate flows for one action's `inputs.ref` consumed at four lines — so re-reads of a root symbol each report. But it reports only one flow for `tj-actions/branch-names`, whose later steps read `steps.branch.outputs.*`. The distinction is not 'first sink wins':

- **Root** symbols (a direct `github.*` reference, or an input bound from `with:` or a default) report at every consumption.
- **Derived** values — those produced by a step that already reported a sink, crossing `$GITHUB_OUTPUT`, `$GITHUB_ENV`, or an `outputs` block — collapse to the first sink.
- Sibling consumptions of the same derived value still each report: gold gives sample 63dd950d both of its two lines, because the value came from a JS action that never reported a sink.

4. Task 2 — Repair

The repair is **env-hoisting**: move the untrusted expression out of the shell script body and into the step's `env:` block, then reference it as an ordinary shell variable. The runner sets the variable in the process environment

rather than pasting it into the source, so the value becomes data again.

```
- run: echo "branch: ${ github.head_ref }"

+ env:
+   HEAD_REF: ${ github.head_ref }
+ run: echo "branch: ${HEAD_REF}"
```

This is not a stylistic choice: env-hoisting accounts for **36 of 36** consumption points across the reference patches, with no exceptions. That is also why the tool needs no language model — the repair is a template, and the hard part is knowing precisely where to apply it.

4.1 Patch scope follows the file, not the sink

The reference patches reveal the scoping rule. For `tj-actions/branch-names`, the reference patches `action.yml` at lines 42, 61 and 75 — including downstream re-consumptions *inside* the action — but leaves the calling workflow's lines 30 and 31 untouched. Sample 63dd950d is the mirror image: only the workflow is patched, because its sinks live there.

So the unit of repair is the *file*, not the individual sink: repair every file containing a reported flow sink, neutralize every consumption inside such a file, and leave a caller's downstream re-consumption alone — neutralizing the root already kills the flow. Encoding this took file-set agreement with the reference patches from 27/29 to **29/29**.

4.2 Self-verification

Every generated patch passes four gates before it is emitted:

Gate	Check
1	git apply --check — the diff is well-formed and applies cleanly
2	Apply to a scratch copy of the tree
3	The patched YAML still parses; actionlint reports no new errors
4	Re-run the detector on the patched tree — require zero surviving flows

Gate 4 is the one that matters. It closes the loop: the tool must defeat its own detector, which makes 'the patch looked plausible' an insufficient outcome.

5. What the verification loop caught

The project also carries a synthetic fixture, but that fixture is *circular* — its cases and its expected answers were written from the same model, so it can only confirm self-consistency. It cannot find a bug it has no shape for. Gate 4, running against real data, found two that it could not:

- **Missing trailing newline (sample 63dd950d).** The file does not end in a newline, and its last line *is* a sink. Python's `diff` then yields a final chunk with no line terminator, so a naive join welds the following `+` line onto the preceding `-` line and git rejects the patch as corrupt. Fixed by emitting git's `\ No newline at end of file` marker.
- **Inline run steps (sample 63dd950c).** A step written `- run: |` carries the `run:` key on the list-item line itself. There is no `run:` line to insert before, so the `env:` block was being inserted *into the shell script*, and the step indent was inferred from the script body rather than the item. Fixed by detecting inline-run steps.

Both are now regression-tested. Neither is exotic; both were simply outside the fixture's imagination, which is the argument for verifying against reality rather than against one's own expectations.

6. Negative results

Two plausible hypotheses were tested and abandoned. Both are reported because the reasoning constrains the model.

6.1 Quoted heredocs are not safe sinks

It is tempting to treat a quoted-delimiter heredoc (`cat << 'EOF'`) as a safe sink: the shell performs no expansion inside the body, so the interpolated text is literal data. The rule is **unsound**. An attacker whose value contains a line equal to the delimiter closes the heredoc early, and everything after it is executed as shell — an EOF breakout. Gold agrees: it flags an action that places a multi-line issue body inside `<< 'EOF'`. Implemented, the rule cost 2 true positives to remove 1 false positive (F1 0.941 to 0.931) and was reverted.

6.2 The trigger does not gate the labels

A natural theory for the remaining false positives is that gold only counts flows reachable under a privileged trigger such as `pull_request_target`. It does not: 16 gold-vulnerable samples use plain `pull_request`, two of them with the same untrusted source as the false-positive sample. The theory is dead, and the tool does not condition on triggers.

7. Limitations

- **The remaining 7 false-positive flows are confined to 2 samples**, and both look like genuine injections under manual analysis. The dataset contains 29 vulnerable samples and exactly 29 reference patches, which suggests labeling covered only flows with a known upstream fix. This is an inference about the labels, not a proven fact.
- **Parts of the model are fit to few examples**. Most notably, gold's `from` line for an input-default flow is the first line in the input's declaration block that mentions the context path — which in one action is the `description`: prose, not the `default`: itself. That is an artifact of label generation, confirmed on 3 of 3 available cases and isolated in a single function rather than spread through the model. Three cases is not many.
- **Recall 1.000 is measured on labels the model was refined against**. The honest check is the held-out set. The strongest independent evidence is Task 2: 29/29 file-set agreement with reference patches the model was never tuned on.
- JavaScript and Docker actions are treated as opaque: inputs are assumed to reach declared outputs. This is deliberately conservative and cannot miss a flow, but it can over-approximate.

8. Reproduction

```
pip install -r requirements.txt
python -m pytest tests/ -q      # 39 unit tests
python eval.py --self-test      # harness sanity: oracle 1.0, null 0.0

# full run (detection + patches + verification)
python -m ghactaint.cli --root DATASET --split test --out .
# -> test.csv + patches/<sample_id>.patch

# score against gold
python eval.py --gold train.csv --pred out/train.csv
```

Dependencies: `ruamel.yaml` (and `pytest` for tests). `actionlint` is optional; verification gate 3 skips cleanly if it is absent. The eval harness is self-testing: it scores a synthetic oracle at 1.0, the null baseline at 0.0, and a deliberately line-shifted prediction at strict 0.0 / file-only 1.0.

9. LLM and tool disclosure

The submitted pipeline makes zero LLM calls. No model is invoked at any point during detection, repair, or verification; no model identifier applies; no API key is required to run the code. The repair is deterministic because env-hoisting covers all 36 reference consumption points, so a language model would add nondeterminism without adding capability — an unattractive trade when the organizers re-run the code on hidden data.

During development, an AI assistant (Anthropic Claude) was used as a programming aid: writing and refactoring code, analyzing the dataset, and drafting this document. All empirical claims here were produced by running the tool and its evaluation harness on the training data, and every design decision above is supported by a measurement recorded in the repository README.

Third-party libraries: `ruamel.yaml` (YAML round-trip), `pytest` (tests), and optionally `actionlint` (workflow linting, verification gate 3). No other external services are contacted at runtime.